Sim-to-Real Transfer for JetBot via Domain Randomization and Recurrent Policies

Florian Tanguy

Semester project

Under the supervision of:
Tingting Ni, Kai Ren and Prof. Maryam Kamgarpour

February 4th, 2026

École Polytechnique Fédérale de Lausanne (EPFL)
1015 Lausanne, Switzerland

Abstract

The deployment of Deep Reinforcement Learning (RL) policies on low-cost mobile robotics platforms is fundamentally constrained by the "Sim-to-Real" gap, i.e. the discrepancy between the idealized physics of simulation and the stochastic, non-linear dynamics of the physical world. While recent advances in Constrained Markov Decision Processes (CMDPs) have enabled the training of safety-aware navigation policies in simulation, these policies can exhibit brittle performance when transferred to physical hardware. This fragility is pronounced in differential-drive robots like the NVIDIA JetBot on which this project is based, where unmodeled actuator deadzones, electromechanical latency, and variable surface friction lead to immobilization and navigational failure.

This project investigates and addresses key causes of Sim-to-Real failure for the JetBot platform, specifically targeting the "stuck robot" phenomenon observed on low-friction surfaces. Building upon previous work in safe RL, we propose a comprehensive pipeline that integrates high-fidelity physics modeling with data-driven domain randomization.

First, we migrate the simulation infrastructure to a fully differentiable, JAX-based environment. This transition facilitates massive parallelization, allowing for the training of agents across thousands of physics variations simultaneously. Second, we address the limitations of standard Uniform Domain Randomization (UDR). By performing System Identification (SysID) on the physical robot, we demonstrate that real-world surface dynamics are multimodal rather than continuous. We consequently implement a Data-Driven randomization strategy using Gaussian Mixture Models (GMM) to accurately represent the disparate regimes of "high-friction/low-deadzone" and "low-friction/high-deadzone" interactions. Finally, to overcome the partial observability induced by wheel slip, where the agent perceives motion despite being stationary, we transition from memoryless Feedforward architectures to Recurrent Neural Networks (LSTMs).

Our results show that while baseline policies trained with Uniform DR transfer successfully to dry, high-friction terrain, they fail on slippery surfaces. The proposed approach, combining Data-Driven physics sampling with recurrent memory, demonstrates significantly improved resilience across heterogeneous real-world environments. While inconsistent behaviors persist in specific deployment configurations, the method effectively narrows the reality gap for cost-constrained robotic hardware.

Contents

1 Introduction **4**
1.1 Motivation: The Reality Gap in Low-Cost Robotics . . . 4
1.1.1 The Sensitivity of Differential Drive Platforms . . . 4
1.1.2 The "Stuck Robot" Phenomenon . . . 5
1.2 Problem Statement . . . 6
1.2.1 Limitations of Uniform Domain Randomization . . . 6
1.2.2 The Observability Challenge: Perceptual Aliasing . . . 6
1.3 Objectives . . . 7
1.4 Contributions . . . 7
1.4.1 JAX-Based Randomization Infrastructure . . . 7
1.4.2 Data-Driven Physics Integration . . . 7
1.4.3 Memory-Based Robustness Analysis . . . 8
1.5 Report Structure . . . 8

2 Background and Related Work **9**
2.1 Reinforcement Learning Fundamentals . . . 9
2.1.1 Markov Decision Processes (MDP) . . . 9
2.1.2 Safety via Constrained MDPs (CMDP) . . . 9
2.1.3 Proximal Policy Optimization (PPO) . . . 10
2.2 The Sim-to-Real Transfer Problem . . . 10
2.2.1 Taxonomy of Gaps . . . 10
2.3 Domain Randomization (DR) . . . 10
2.3.1 Uniform Domain Randomization (UDR) . . . 11
2.3.2 Data-Driven Domain Randomization . . . 11
2.4 Unobserved Dynamics and Non-Markovian Policies . . . 11
2.4.1 The Hidden Physics Problem . . . 11
2.4.2 State Recovery via Recurrent Neural Networks . . . 12

3 System Overview and Simulation Environment **13**
3.1 Simulation Infrastructure: The Migration to JAX . . . 13
3.1.1 Limitations of the Previous Architecture . . . 13
3.1.2 The JAX-Based Environment . . . 14
3.2 The Physical Platform: NVIDIA JetBot . . . 14
3.3 Mathematical Physics Modeling . . . 14
3.3.1 Differential Drive Kinematics . . . 14
3.3.2 Actuator Model: Piecewise Affine Functions . . . 15
3.3.3 Action Space Constraints (Model Validity) . . . 15

3.4 System Identification and Parameter Extraction . . . 16
3.4.1 Raw Data Analysis . . . 16
3.4.2 The "Zero-Slope" Problem . . . 16
3.4.3 Refinement Strategy . . . 17
3.4.4 Final Parameter Extraction . . . 17
3.5 Data-Driven Domain Randomization Strategies . . . 18
3.5.1 Strategy 1: Uniform Domain Randomization (Baseline) . . . 18
3.5.2 Strategy 2: Parametric Gaussian Mixture Model (GMM) . . . 18
3.5.3 Strategy 3: Non-Parametric Direct Data Replay . . . 19

4 Reinforcement Learning Framework **20**
4.1 Problem Formulation . . . 20
4.1.1 Observation Space $\mathcal{S}$ . . . 20
4.1.2 Action Space $\mathcal{A}$ . . . 21
4.1.3 Reward Function r . . . 21
4.1.4 Cost Function c and Safety Constraints . . . 22
4.2 Policy Architecture: Memory and State Estimation . . . 22
4.2.1 The Perceptual Aliasing Problem . . . 22
4.2.2 Approach I: Frame Stacking (Finite Context) . . . 22
4.2.3 Approach II: Recurrent PPO (Infinite Context) . . . 23
4.2.4 Policy Interpretation: Implicit Context Adaptation . . . 23
4.2.5 Training Hyperparameters . . . 23
4.3 Logic-Based Safety Wrappers . . . 23

5 Experiments and Results **25**
5.1 Experimental Setup . . . 25
5.1.1 Simulation Performance (The Reality Gap) . . . 25
5.1.2 Real-World Scenarios . . . 25
5.2 Phase I: The Baseline (MLP + Uniform Randomization) . . . 26
5.2.1 Performance on Dry Ground . . . 26
5.2.2 Failure Mode: The "Spinning" Robot . . . 26
5.3 Phase II: Adding Temporal Context (Frame Stacking) . . . 27
5.3.1 Behavioral Analysis . . . 27
5.4 Phase III: Recurrent Policy (LSTM) . . . 28
5.4.1 Emergent Recovery Behavior (Adaptive Re-planning) . . . 28
5.4.2 Stability and Inconsistency . . . 29
5.5 Ablation Study: Architecture and Observation Space . . . 30
5.5.1 Implicit vs. Explicit Action History . . . 30
5.6 Analysis of Physics Modeling (GMM vs. Data-Driven) . . . 30
5.7 Summary of Experimental Findings . . . 31

6 Conclusion and Future Work **32**
6.1 Summary of Contributions . . . 32
6.2 Limitations . . . 33
6.3 Future Work . . . 33
6.3.1 Explicit Meta-Reinforcement Learning . . . 33
6.3.2 Online System Identification . . . 33
6.4 Closing Remarks . . . 33

Chapter 1

Introduction

Autonomous mobile navigation in unstructured environments remains a cornerstone challenge in robotics. The ability for a robot to perceive its surroundings, plan a trajectory, and execute control commands while avoiding obstacles is a prerequisite for applications ranging from warehouse logistics and search-and-rescue to domestic service robots. Traditional approaches to this problem, such as Simultaneous Localization and Mapping (SLAM) combined with classical path planning algorithms (e.g., A*, Dynamic Window Approach), have proven effective but often rely on expensive sensor suites and computationally intensive optimization routines. Furthermore, these classical methods often struggle to adapt to dynamic changes in the environment or unmodeled physical interactions, such as wheel slip on varying terrains.

Deep Reinforcement Learning (DRL) has emerged as a promising alternative, offering an end-to-end learning paradigm where a neural network policy maps raw sensor observations directly to control actions. By optimizing a cumulative reward signal through trial-and-error, DRL agents can theoretically learn complex, robust behaviors that are difficult to manually codify. However, the practical application of DRL to physical robots is severely hindered by the logistical constraints of data collection. DRL algorithms are notoriously sample-inefficient, often requiring millions of interaction steps to converge. Training a robot in the real world is not only time-consuming but also fraught with risks: mechanical wear and tear, battery depletion, and the potential for collisions that could damage the robot or its environment.

Consequently, the standard paradigm in modern robotic learning is "Sim-to-Real" transfer: training agents in a high-speed, safe simulation and then deploying the learned policy onto the physical hardware. While appealing, this approach introduces a critical new challenge: the Sim-to-Real gap.

1.1 Motivation: The Reality Gap in Low-Cost Robotics

The Sim-to-Real gap refers to the performance degradation observed when a policy trained in a virtual environment is deployed in the real world. This gap arises because no simulation, regardless of its fidelity, is a perfect isomorphism of physical reality.

1.1.1 The Sensitivity of Differential Drive Platforms

For high-end robotic platforms (e.g., quadruped robots or industrial arms), high-precision actuators and encoders allow for faithful simulation using standard rigid-body physics engines (e.g., NVIDIA PhysX, MuJoCo). These engines excel at simulating contact forces and inertia

but often abstract the low-level motor control layer, assuming ideal velocity tracking or torque control.

However, for low-cost platforms like the NVIDIA JetBot, this abstraction conceals the primary source of the reality gap. The JetBot utilizes generic DC motors driven by Pulse Width Modulation (PWM) without low-level PID velocity control. This introduces specific electromechanical non-linearities that are computationally expensive or difficult to implement efficiently in general-purpose physics engines:

1. **Actuator Deadzones:** Real DC motors require a minimum activation voltage to overcome static friction. Below this threshold, the input is non-zero, but the output velocity is zero. Standard RL environments often assume a linear relationship where $velocity \propto input$, or use high-fidelity engines that do not expose these specific friction parameters for rapid randomization.

2. **Asymmetric Motor Response:** Due to manufacturing tolerances, the left and right motors rarely possess identical gain curves. A command of $[0.5, 0.5]$ often results in a curved trajectory.

3. **Surface-Dependent Dynamics:** The interaction is not just Coulomb friction but a complex function of wheel slip and voltage sag. While engines like PhysX can model complex friction, tuning these parameters to match the stochastic behavior of a cheap robot on varying terrain is non-trivial and hinders the throughput required for large-scale Domain Randomization.

Thus, we migrated to a simplified JAX-based environment not for superior general physics fidelity, but to gain complete, differentiable control over the *specific* actuator dynamics and friction models relevant to the JetBot, enabling massive parallelization that would be intractable in heavier simulations.

1.1.2 The "Stuck Robot" Phenomenon

A specific motivation for this work arises from the Sim-to-Real challenges identified in the Sycamore Lab's previous semester project ("Safe Reinforcement Learning for Robot Maze Escaping", Rocca Federico, [3]). While that work successfully implemented Lagrangian PPO to enforce safety constraints in simulation, the transfer to the physical robot proved difficult. Upon deployment, the policies exhibited significant *instability and erratic behavior*, failing to consistently respect obstacles or maintain coherent trajectories on the real hardware.

Initial efforts in this project utilizing standard Uniform Domain Randomization (UDR) successfully bridged this general gap, enabling stable navigation on standard high-friction surfaces (lab's ground, sandpaper, dry plastic). However, when the operational domain was expanded to include **low-friction surfaces** (e.g., wet terrain or puddles), a new, persistent failure mode emerged: *immobilization*.

In these slippery conditions, the robot would frequently enter a state where it commanded forward throttle but failed to overcome the specific combination of static friction (deadzone) and wheel slip. Crucially, because the baseline policy was memoryless (Markovian), it could not distinguish between "moving forward" and "slipping in place." This specific inability to generalize to the lower bound of the friction spectrum, despite success on dry ground, highlights the necessity for the advanced Data-Driven and Memory-based strategies proposed in this report.

1.2 Problem Statement

The central problem addressed in this project is the lack of robustness in DRL navigation policies when transferred to physical environments with heterogeneous and unmodeled dynamics. Specifically, we address the failure of standard Domain Randomization techniques to prepare agents for the non-linear "deadzone" and "slip" regimes encountered on real-world low-friction surfaces.

1.2.1 Limitations of Uniform Domain Randomization

Domain Randomization (DR) is the standard technique for bridging the reality gap. It involves randomizing simulation parameters (mass, friction, motor strength) during training to force the agent to learn a policy that is robust to variations. The prevailing approach, Uniform Domain Randomization (UDR), samples these parameters from a uniform distribution (e.g., friction $\mu \sim U[0.5, 1.0]$).

We hypothesize that UDR is insufficient for the JetBot platform for two reasons:

1. **Distribution Mismatch:** The physical world is not uniform. A floor is typically either "dry/grippy" or "wet/slippery." Training on a uniform average dilutes the agent's exposure to the specific, difficult edge cases (high deadzone, low friction) that cause failure.

2. **Implausible Physics:** Naively randomizing parameters without considering their correlation (e.g., high friction often correlates with lower effective deadzone) can generate "impossible" simulation environments that hinder learning or lead to overly conservative policies.

1.2.2 The Observability Challenge: Perceptual Aliasing

The second aspect of the problem is architectural and stems from the specific sensor constraints of our experimental setup.

To simulate a low-cost blind platform, we operate the JetBot without wheel encoders or visual odometry. The agent is provided with a sparse set of range-measurements (simulating distance sensors) but has no direct measurement of its linear or angular velocity. To prevent the policy from overfitting to noisy derivative estimates, we explicitly removed velocity from the observation space.

This creates a Violation of the Markov Assumption.

- **The Blindness:** The agent observes only the current geometrical distance to obstacles and the goal vector. It lacks proprioceptive feedback regarding its true motion.

- **Perceptual Aliasing:** Consider a robot stuck on a slippery patch. It applies throttle ($a_t > 0$), but due to friction, it remains stationary. The distance readings o_t remain static. To a memoryless Feedforward policy, this state is indistinguishable from a robot that has simply stopped moving.

Because the instantaneous observation o_t is insufficient to distinguish between "static" and "stuck," the agent must infer its motion state from the *temporal evolution* of its inputs.

1.3 Objectives

The overarching goal of this project is to develop a robust Sim-to-Real transfer pipeline that enables a JetBot to navigate safely and effectively across varying terrain types, specifically overcoming the "stuck" failure mode on low-friction surfaces.

This is decomposed into the following specific milestones:

- **Objective 1: High-Fidelity Physics Simulation.** Migrate the existing training infrastructure to a framework capable of modeling non-linear actuator dynamics. This involves implementing a differentiable or high-throughput physics engine (JAX) that explicitly simulates the motor deadzone and the high-gain response curve identified in the real hardware.
- **Objective 2: Data-Driven Domain Randomization.** Move beyond uniform parameter sampling. We aim to perform System Identification (SysID) on the physical robot to collect real-world interaction data. This data will be used to construct a Ground Truth distribution, specifically using Gaussian Mixture Models (GMM), to ensure the simulation training distribution matches the bimodal nature of the real world.
- **Objective 3: Temporal Robustness via Recurrence.** Investigate the efficacy of Recurrent Neural Networks (specifically LSTMs) in solving the "stuck" problem. By providing the agent with a hidden state vector representing memory, we aim to enable emergent recovery behaviors (e.g., "wiggling" or changing direction) when forward motion is impeded.
- **Objective 4: Real-World Validation.** Validate the proposed methods not just in simulation, but on the physical JetBot. The success metric is qualitative robustness: the ability of the robot to navigate a course on both dry (high friction) and wet/slippery (low friction) surfaces without requiring manual intervention.

1.4 Contributions

This project makes several technical contributions to the Sycamore Lab's robotics framework:

1.4.1 JAX-Based Randomization Infrastructure

We successfully migrated the training pipeline from a CPU-bound PyTorch environment to a hardware-accelerated JAX environment. This migration unlocks massive parallelization, allowing us to simulate thousands of independent environments on a single GPU. This throughput is critical for Domain Randomization, as it allows the agent to experience millions of permutations of physics parameters in a feasible timeframe.

1.4.2 Data-Driven Physics Integration

We integrated the Sycamore Lab's Piecewise Affine Actuator Model (which explicitly accounts for deadzones and gain) into the differentiable JAX environment. By combining this high-fidelity model with real-world System Identification data, we developed a Data-Driven Domain Randomization strategy. Unlike standard methods that sample from uniform distributions, our approach samples directly from the multimodal distributions observed in reality (e.g., distinct "Wet" vs. "Dry" clusters).

1.4.3 Memory-Based Robustness Analysis

We conducted a rigorous comparative analysis of architectural solutions to the "Stuck Robot" problem. We evaluated:

1. **Project Baseline:** Uniform Domain Randomization (Memoryless).
2. **Frame Stacking:** Sliding window memory (Finite Context).
3. **Recurrent Policies (LSTM):** Hidden state memory (Indefinite Context).

We demonstrate that while Frame Stacking can resolve short-term ambiguities (estimating instantaneous velocity), it scales poorly for long-duration immobilization events due to input dimensionality constraints. The integration of LSTM memory proves significantly more robust, providing the long-horizon temporal context required to detect and recover from persistent stuck states.

1.5 Report Structure

The remainder of this report is organized as follows:

Chapter 2: Background and Related Work provides an overview of the theoretical foundations of Reinforcement Learning, the specific challenges of Sim-to-Real transfer, and a review of relevant literature in Domain Randomization and Recurrent Neural Networks.

Chapter 3: Methodology - Simulation and Physics details the construction of the JAX-based simulation environment. It presents the mathematical formulation of the piece-wise affine actuator model, the System Identification process used to gather real-world friction data, and the implementation of Data-Driven Domain Randomization.

Chapter 4: Methodology - Learning and Architecture describes the RL framework. It details the transition from memoryless Feedforward networks to Recurrent PPO (LSTM), compares these against Frame Stacking approaches, and explains the reward formulation and logic-based safety wrappers.

Chapter 5: Experiments and Results presents the experimental validation. We provide a qualitative analysis of the deployment on the physical JetBot, focusing on the resolution of the "stuck robot" failure mode.

Chapter 6: Conclusion summarizes the findings, discusses the limitations of the current approach (such as inconsistent behaviors), and suggests directions for future research.

Chapter 2

Background and Related Work

This chapter establishes the theoretical foundations for the project. We briefly review the problem formulation based on Constrained Markov Decision Processes (CMDPs) utilized in the underlying framework, and the Proximal Policy Optimization (PPO) algorithm used for training. We then focus on the core literature relevant to our specific contributions: Domain Randomization and Recurrent Neural Networks for Sim-to-Real transfer.

2.1 Reinforcement Learning Fundamentals

2.1.1 Markov Decision Processes (MDP)

The standard formulation for robotic navigation is the Markov Decision Process (MDP), defined as a tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$.

- $\mathcal{S} \subseteq \mathbb{R}^n$: The state space (pose, goal vector).
- $\mathcal{A} \subseteq \mathbb{R}^m$: The action space (motor voltage commands).
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \to \mathcal{P}(\mathcal{S})$: The transition dynamics $P(s_{t+1}|s_t, a_t)$. In this work, accurately modeling $\mathcal{P}$ to match reality is the primary objective.
- $\mathcal{R}$: The reward function encouraging goal-reaching efficiency.

2.1.2 Safety via Constrained MDPs (CMDP)

To ensure the robot operates safely (e.g., avoiding collisions), the problem is extended to a Constrained MDP (CMDP). In addition to maximizing reward, the agent must satisfy a set of cost constraints:

$$\max_{\pi} J(\pi) \quad \text{s.t.} \quad J_C(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_t \gamma^t C(s_t, a_t) \right] \leq d \tag{2.1}$$

where $C(s, a)$ is a collision cost indicator and d is a safety threshold.

Note on Implementation: This project builds upon the **Lagrangian-PPO** framework developed in prior Sycamore Lab research [3]. That work focused on solving the constrained optimization via primal-dual updates. In this project, we adopt this safety framework as a fixed backbone. Our contributions focus strictly on the robustness of the transition dynamics $\mathcal{P}$ (Sim-to-Real) rather than the constraint satisfaction mechanism itself.

2.1.3 Proximal Policy Optimization (PPO)

The policy is trained using Proximal Policy Optimization (PPO) [4]. PPO is chosen for its stability in environments with high variance (such as those introduced by Domain Randomization). The core mechanism is the clipped surrogate objective, which prevents destructive large policy updates:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{2.2}$$

where $r_t(\theta)$ is the probability ratio between new and old policies, and ϵ is the clipping parameter (typically 0.2). This objective allows the agent to safely exploit the data generated by the randomized physics engine.

2.2 The Sim-to-Real Transfer Problem

The "Sim-to-Real gap" can be formalized as a problem of distribution shift. Let the source domain (simulation) be characterized by a distribution over MDPs, $\mathcal{D}_{sim}$, and the target domain (reality) be a single, specific MDP formulation $\mathcal{M}_{real}$ (or a distribution $\mathcal{D}_{real}$ representing the changing real world).

The fundamental issue is that $\mathcal{D}_{sim} \neq \mathcal{D}_{real}$. Specifically, the transition dynamics differ:

$$P_{sim}(s_{t+1}|s_t, a_t) \neq P_{real}(s_{t+1}|s_t, a_t) \tag{2.3}$$

In the context of the JetBot, this inequality stems from unmodeled friction coefficients, motor deadzones, and communication latencies.

2.2.1 Taxonomy of Gaps

The Sim-to-Real gap is generally categorized into three components:

1. **Dynamics Gap:** Mismatches in physical parameters (mass, inertia, friction) and actuator models. This is the primary focus of this project (e.g., deadzone modeling).
2. **Perception/Latency Gap:** While the geometrical layout of the map is known, the physical system introduces delays. The state observed by the agent s_t corresponds to the physical state of the robot at time $t - \Delta t_{comm}$, whereas the simulation is instantaneous.
3. **Control Gap:** The discrete time-stepping of the simulation does not perfectly capture the continuous voltage degradation and PWM frequency effects of the real motor drivers.

2.3 Domain Randomization (DR)

Domain Randomization (DR) is arguably the most successful technique for zero-shot Sim-to-Real transfer. The core insight, introduced by Tobin et al. [5], is to randomize the parameters of the simulation ξ (where ξ includes friction, mass, color, lighting, etc.) across a wide distribution Ξ.

Instead of optimizing the policy for a single environment $\mathcal{M}$, we maximize the expected return over the distribution of environments induced by Ξ:

$$\theta^* = \arg\max_\theta \mathbb{E}_{\xi \sim \Xi} \left[\mathbb{E}_{\tau \sim \pi_\theta, \mathcal{M}(\xi)} \left[\sum r_t \right] \right] \tag{2.4}$$

If the distribution Ξ is sufficiently wide and covers the true parameters of the real world ($\xi_{real} \in \text{supp}(\Xi)$), the resulting policy should be robust to the real dynamics.

2.3.1 Uniform Domain Randomization (UDR)

In its simplest form, UDR assumes a uniform prior over the physics parameters. For example, the coefficient of dynamic friction might be sampled as $\mu \sim U(0.1, 1.0)$. While UDR has shown success in rigid-body manipulation tasks (e.g., OpenAI's Dactyl hand), it has limitations for mobile robotics:

- **Conservatism:** To cover all possibilities, the ranges must be large. This often produces "impossible" environments (e.g., zero friction) where no policy can succeed, destabilizing training.
- **Mean-Reversion:** The policy effectively learns to navigate the "average" environment. If the real world lies at the tails of the uniform distribution (e.g., extremely slippery wet floor), the "average" strategy may fail.

2.3.2 Data-Driven Domain Randomization

To address the inefficiencies of UDR, researchers have proposed Guided or Adaptive DR [1], which typically involves iteratively updating simulation distributions during training.

In this project, we adopt a specific variant of *Data-Driven Domain Randomization* relying on offline System Identification. To accurately represent the real-world dynamics without the limitations of uniform assumptions, we investigate two distinct strategies for utilizing the collected data:

1. **Parametric Sampling (GMM):** We fit a Gaussian Mixture Model (GMM) to the real-world data. This approach constructs a continuous probability density function that captures the multi-modal nature of the terrain (e.g., clustering "slippery" vs. "grippy" modes) while allowing for generalization around those modes.
2. **Non-Parametric Sampling (Direct Replay):** We bypass distribution fitting and sample physics parameters directly from the raw System Identification dataset. This ensures the simulation utilizes "Ground Truth" configurations, preserving the exact noise profiles and parameter correlations observed in reality without the bias introduced by a fitted model.

2.4 Unobserved Dynamics and Non-Markovian Policies

A standard Markov Decision Process (MDP) assumes that the current state s_t fully summarizes the environment, such that the transition probability $P(s_{t+1}|s_t, a_t)$ depends only on the immediate present. However, in our "stuck robot" scenario, this assumption is violated because the physical parameters governing motion are not observable.

2.4.1 The Hidden Physics Problem

Consider a robot navigating a surface with variable friction. The visible state s_t (distance rays and goal vector) might be identical for two different scenarios:

1. **High Friction:** Action $a_t > 0 \implies$ Robot accelerates.
2. **Low Friction:** Action $a_t > 0 \implies$ Robot slips (velocity ≈ 0).

In a fully equipped robot, this ambiguity would be resolved by reading wheel encoders (odometry). In our setup, although the MoCap system provides global position (x, y), estimating instantaneous velocity via finite differencing $(\Delta x/\Delta t)$ amplifies high-frequency tracking noise. Furthermore, relying on theoretical wheel speeds derived from voltage models creates a significant Sim-to-Real gap, as the real robot's velocity profile under load differs from the idealized model. Consequently, we removed explicit velocity terms from the observation space to force the agent to rely on temporal features rather than unreliable instantaneous derivatives.

Without velocity feedback, the friction coefficient μ becomes a latent variable. From the agent's perspective, the environment becomes Non-Markovian: the optimal action cannot be determined solely from the current observation, but requires knowledge of the interaction history.

2.4.2 State Recovery via Recurrent Neural Networks

To recover a quasi-Markovian state representation, we employ Recurrent Neural Networks (RNNs). Specifically, the Long Short-Term Memory (LSTM) network maintains a hidden state vector h_t that aggregates the history of observations:

$$h_t = \text{LSTM}(o_t, h_{t-1}) \tag{2.5}$$

This hidden state h_t serves as a proxy for the unobserved physical dynamics. By integrating the discrepancy between the commanded action and the resulting (static) observation over time, the LSTM allows the policy $\pi(a_t|h_t)$ to distinguish between a robot that has just stopped and a robot that is persistently stuck, enabling the emergence of recovery behaviors without requiring direct velocity sensors.

Chapter 3

System Overview and Simulation Environment

This chapter details the technical architecture of the simulation environment developed for this project. While standard Reinforcement Learning (RL) platforms like NVIDIA Isaac Lab provide powerful, GPU-accelerated rigid body physics, they are designed primarily for general-purpose contact dynamics. Integrating specific, non-smooth electromechanical models, such as voltage-dependent motor deadzones, into these pre-compiled engines often introduces significant implementation complexity and overhead. Consequently, to achieve the granular control over actuator dynamics required for our Sim-to-Real strategy, we developed a custom, end-to-end compiled simulation infrastructure using JAX.

We first describe the motivation and technical advantages of migrating to this JAX-based framework, specifically focusing on programmability and throughput. We then present the mathematical derivation of the "Piecewise Affine" differential drive model, utilizing the Sycamore Lab's formulation to accurately represent motor non-linearities. Finally, we detail the System Identification (SysID) process used to capture real-world dynamics and the subsequent Data-Driven Domain Randomization strategies employed to model surface friction.

3.1 Simulation Infrastructure: The Migration to JAX

3.1.1 Limitations of the Previous Architecture

The previous iteration of this project [3] utilized NVIDIA Isaac Lab, which leverages the PhysX 5 engine for rigid body dynamics. While Isaac Lab supports massive parallelization on the GPU (simulating 4,096+ environments), it relies on a standard high-fidelity physics solver designed for general-purpose contacts and friction.

However, for Sim-to-Real transfer on the JetBot, the primary source of the reality gap is not rigid-body contact dynamics, but the specific electromechanical response of the actuators (as detailed in Section 3.3.2). Implementing a custom, non-smooth actuator model (e.g., a parameterized deadzone function) inside a pre-compiled engine like PhysX is non-trivial. It often requires complex plugin development or allows modification of global parameters (like friction coefficients) rather than the structural equations of motion for individual agents. This rigidity limits the scope of Domain Randomization to standard parameters supported by the API.

3.1.2 The JAX-Based Environment

To address this, we developed a custom simulation environment using JAX. While JAX provides similar GPU-accelerated parallelization to Isaac Lab, its primary advantage for this project is programmability.

In JAX, the entire environment step—including the custom motor dynamics, collision logic, and reward calculation—is defined as a pure function. This offers two decisive advantages:

1. **Customizable Dynamics:** We can directly inject the Sycamore Lab's "Piecewise Affine" model (Eq. 3.3) into the physics kernel. This allows us to randomize the *structure* of the actuation (e.g., varying deadzone thresholds δ per environment) as easily as standard parameters.

2. **End-to-End Compilation (XLA):** Using `jax.jit`, the physics integration and the RL policy are compiled into a single fused CUDA kernel. This eliminates the Python interpreter overhead entirely during the training loop, maximizing throughput for our lightweight 2D maze environment.

Thus, the migration was motivated by the need to efficiently randomize the *structure* of the actuation model, rather than raw throughput alone.

3.2 The Physical Platform: NVIDIA JetBot

The target hardware is the NVIDIA JetBot, a low-cost differential drive robot. For the purposes of this study, the critical electromechanical characteristic is its open-loop control scheme.

Unlike industrial robots equipped with quadrature encoders and PID velocity controllers, the JetBot's motors are driven directly by Pulse Width Modulation (PWM) voltage. Consequently, the RL agent's action a_t corresponds to a voltage ratio rather than a target velocity. This absence of low-level velocity regulation means the robot's motion is entirely subject to surface friction and motor non-linearities, necessitating the rigorous physics modeling detailed in the following section.

3.3 Mathematical Physics Modeling

To bridge the Sim-to-Real gap, we model the JetBot as a differential drive system subject to specific electromechanical non-linearities. We adopt the **Piecewise Affine Control Model** developed by the Sycamore Lab to accurately represent actuator behavior.

3.3.1 Differential Drive Kinematics

The state of the robot is defined by its pose $\mathbf{s} = [x, y, \theta]^T$. The control model relates the linear velocity v and angular velocity ω to the state evolution:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v\cos(\theta) \\ v\sin(\theta) \\ \omega \end{bmatrix} \tag{3.1}$$

The individual wheels are linked to the robot's motion model as follows:

$$v = \frac{R}{2}(\omega_r + \omega_l), \quad \omega = \frac{R}{L}(\omega_r - \omega_l) \tag{3.2}$$

where ω_r, ω_l are the rotation rates of the right and left wheels, R is the wheel radius, and L is the baseline distance between wheels.

3.3.2 Actuator Model: Piecewise Affine Functions

In standard simulations, wheel speed is often modeled as proportional to input ($\omega \propto u$). However, real DC motors exhibit significant deadzones due to static friction.

The mapping from control input $u \in [-1, 1]$ to wheel rotation rate ω is defined independently for the right (r) and left (l) motors:

$$\omega_r = F_r(u_r) = \begin{cases} \alpha_{r-}(u_r - d_{r-}) & \text{if } u_r < d_{r-} \\ 0 & \text{if } d_{r-} \leq u_r \leq d_{r+} \\ \alpha_{r+}(u_r - d_{r+}) & \text{if } u_r > d_{r+} \end{cases} \tag{3.3}$$

$$\omega_l = F_l(u_l) = \begin{cases} \alpha_{l-}(u_l - d_{l-}) & \text{if } u_l < d_{l-} \\ 0 & \text{if } d_{l-} \leq u_l \leq d_{l+} \\ \alpha_{l+}(u_l - d_{l+}) & \text{if } u_l > d_{l+} \end{cases} \tag{3.4}$$

Here, d represents the **Deadzone parameters** (thresholds for motion) and α represents the **Gain parameters** (slope of the motor curve).

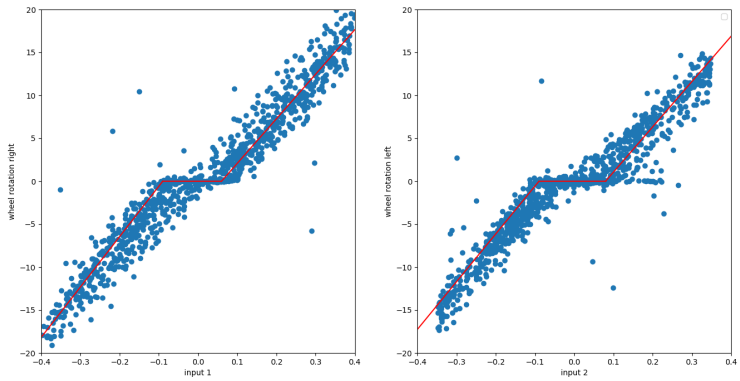

Figure 3.1: **Actuator System Identification:** The blue dots represent empirical velocity measurements from the real JetBot. The red line shows the fitted Piecewise Affine Model. Note the distinct deadzone where input $u \neq 0$ results in $\omega = 0$.

3.3.3 Action Space Constraints (Model Validity)

The Piecewise Affine model parameters (α, d) were derived from System Identification data collected over a specific input range (e.g., $u \in [-0.6, 0.6]$). Extrapolating the linear fit beyond this measured range is risky, as physical motors typically exhibit saturation effects (voltage sag) at higher duty cycles that the affine model fails to capture.

To ensure the simulation remains grounded in empirical data, we explicitly constrain the effective motor commands. Regardless of the raw policy output, the simulation environment

clamps the input passed to the physics model to the validated range limits (e.g., $|u_{eff}| \leq 0.6$). This guarantees that the agent operates strictly within the validated physics regime, preventing it from exploiting unmodeled high-velocity dynamics.

3.4 System Identification and Parameter Extraction

To calibrate the physics parameters α (gain) and δ (deadzone), we analyzed the raw system identification data provided by the Sycamore Lab. The dataset consists of open-loop motor commands $u \in [-0.6, 0.6]$ and the corresponding measured wheel velocities.

3.4.1 Raw Data Analysis

The raw telemetry exhibited significant stochasticity, particularly in the low-velocity regimes (Figure 3.2). While the "Dry" surface data showed a clear linear trend, the "Wet" surface data was dominated by noise and frequent immobilization events (where $u > 0$ but $\omega = 0$).

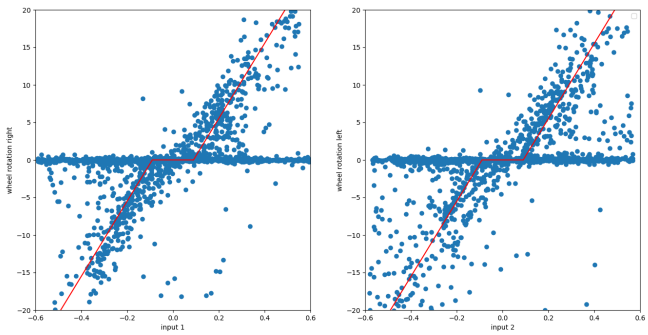

Figure 3.2: **Raw Telemetry Data:** The initial dataset shows a clear bifurcation between high-friction (steep slope) and low-friction (shallow slope) regimes, but is heavily polluted by stationary data points near zero velocity.

3.4.2 The "Zero-Slope" Problem

We initially attempted to fit the Piecewise Affine Model directly to this raw data using a Residual-Based Classification strategy. However, this naive optimization yielded a degenerate result for the Wet regime.

As shown in Figure 3.3, the abundance of "stalled" data points (zero velocity due to static friction) dominated the least-squares error calculation. Consequently, the optimizer minimized error by fitting a nearly flat line ($\alpha \approx 0.0$) through the zero-velocity cluster. Implementing this model would render the simulated robot completely immobile on wet surfaces, effectively deleting the "slip" dynamics we intended to study.

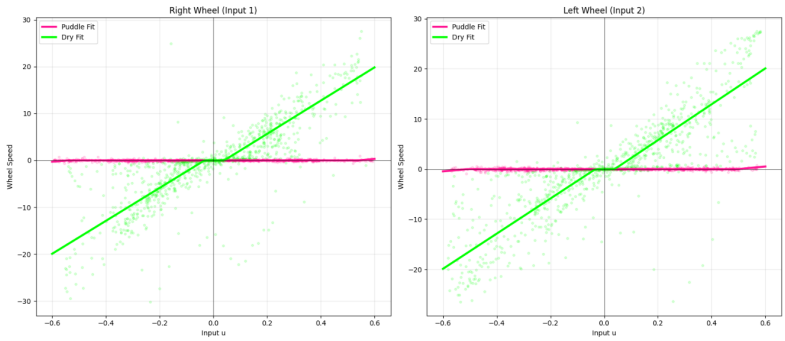

Figure 3.3: **Degenerate Naive Fit:** An initial fit on raw data resulted in a "Puddle" model (Pink) with near-zero slope, as the optimizer prioritized fitting the stalled (zero velocity) points over the moving ones.

3.4.3 Refinement Strategy

To extract a physically useful model from the noisy data, we applied a two-stage refinement process:

1. **Heuristic Filtering:** We reasoned that the "stalled" points, while physical, obscure the kinetic friction relationship we need to simulate motion. We therefore filtered out data points below a noise floor ($|\omega| < 0.1$ rad/s) and removed sign mismatches (e.g., negative velocity for positive input) to isolate the *moving* cases.

2. **Physics-Informed Constraints:** Even after filtering, the separation between modes was ambiguous. We introduced a lower bound to the gain parameter (e.g., enforcing $\alpha_{puddle} > 4.0$) during the fitting process. This forced the solver to capture the gradient of the reduced-traction motion rather than collapsing to the stationary mean.

3.4.4 Final Parameter Extraction

The refined fit (Figure 3.4) successfully decouples the dynamics into two usable simulation regimes:

- **Dry Regime (Green):** High gain ($\alpha \approx 40.0$), negligible deadzone.
- **Puddle Regime (Pink):** Attenuated gain ($\alpha \approx 9.3$). Crucially, this non-zero slope ensures the simulated robot experiences *reduced control authority* (wiggling/slipping) rather than total paralysis.

These distributions (μ, σ) obtained via bootstrapping serve as the ground truth basis for the Domain Randomization defined in Chapter 4.

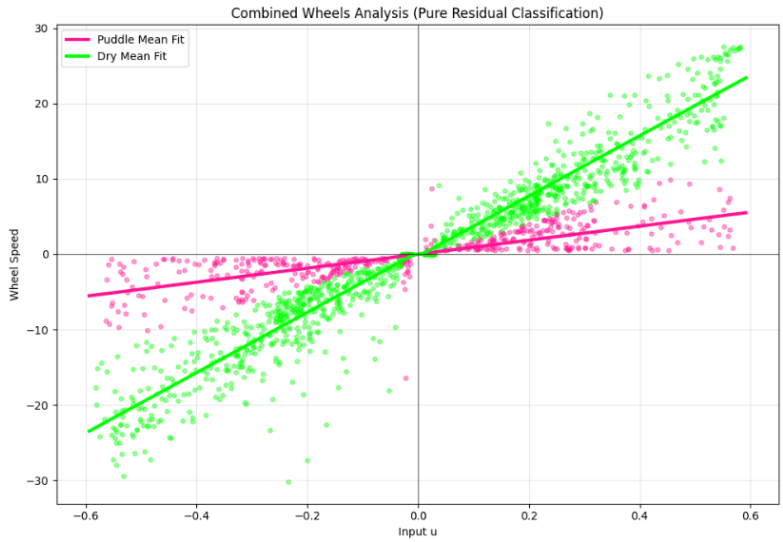

Figure 3.4: **Final Bootstrapped Model:** After filtering stationary points and applying constraints, the model correctly identifies a "Puddle" regime (Pink) that allows for slow, slip-heavy motion.

3.5 Data-Driven Domain Randomization Strategies

To address the Sim-to-Real gap identified in Chapter 3, we implemented three distinct Domain Randomization (DR) strategies. These strategies define how the physics engine samples the transition dynamics $\mathcal{P}(s'|s,a)$ during training.

3.5.1 Strategy 1: Uniform Domain Randomization (Baseline)

The baseline approach represents the standard practice for many robotics tasks: sampling physics parameters from independent uniform distributions. Crucially, this baseline was calibrated under the assumption of standard operating conditions (dry ground), as typically found in educational robotics setups.

$$\delta \sim \mathcal{U}(0.05, 0.20) \tag{3.5}$$

$$\alpha \sim \mathcal{U}(45, 55) \tag{3.6}$$

Hypothesis: This strategy is expected to yield high performance on high-friction surfaces but fail catastrophically in low-friction ("Wet") regimes, as the training distribution does not cover the high-deadzone, low-gain dynamics associated with slip.

3.5.2 Strategy 2: Parametric Gaussian Mixture Model (GMM)

To explicitly target the bimodal nature of the environment (Wet vs. Dry), we utilized the parameters extracted in Chapter 3 to fit a 2-component Gaussian Mixture Model (GMM). Unlike UDR, which samples α and δ independently, the GMM captures the joint probability $P(\alpha, \delta)$, preserving the correlation that high-friction surfaces typically have low deadzones.

During each environment reset, the simulator:
1. Samples a mode $k \in \{\text{Dry}, \text{Wet}\}$ based on a Bernoulli prior (e.g., $P(\text{Wet}) = 0.55$).
2. Samples (α, δ) from the corresponding multivariate normal distribution $\mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

This parametric approach forces the agent to learn a robust policy that can identify and adapt to two distinct "physics clusters."

3.5.3 Strategy 3: Non-Parametric Direct Data Replay

Standard parametric models (like the Piecewise Affine model used in Strategies 1 and 2) rely on the assumption that the motor noise is Gaussian ($\epsilon \sim \mathcal{N}(0, \sigma)$). However, real-world electromechanical noise is often structured, asymmetric, and state-dependent.

To eliminate modeling bias, we implemented a Non-Parametric Direct Replay strategy.

1. **Offline Table Generation:** We discretized the input space $u \in [-1, 1]$ into bins of width 0.05. For each bin, we populated a replay buffer with up to 2,000 raw velocity measurements (ω_L, ω_R) collected from the physical robot.

2. **Online Sampling:** At every simulation step t, instead of computing velocity via an equation $v = \alpha(u - \delta)$, the simulator queries the replay buffer. Given the current action a_t, it retrieves a random real-world velocity sample recorded for that specific input magnitude.

Hypothesis: This method provides the highest fidelity "Ground Truth" simulation, as it reproduces the exact stochasticity, quantization, and outliers of the physical actuators without forcing them through a simplified mathematical model.

Chapter 4

Reinforcement Learning Framework

This chapter details the learning framework employed to train the navigation policies. We define the problem formally as a Constrained Markov Decision Process (CMDP), adopting the notation established in previous Sycamore Lab research [3]. We specify the observation space, action space, and reward structure tailored for the JAX-based environment. Finally, we motivate the architectural transition from standard Feedforward policies to Recurrent Neural Networks (LSTMs) to resolve the partial observability introduced by the "stuck" robot phenomenon.

4.1 Problem Formulation

We model the navigation task as a Constrained Markov Decision Process (CMDP), defined by the tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, r, c, \gamma, \rho \rangle$.

4.1.1 Observation Space $\mathcal{S}$

To maximize sample efficiency in the JAX environment, we adopt a compact, feature-based observation space similar to [3], but adapted to include action history for recurrent processing. The state vector $s_t \in \mathbb{R}^{12}$ is defined as:

$$s_t = (x_t, y_t, d_{G,t}, \Delta\psi_{G,t}, d_{1,t}, d_{2,t}, d_{3,t}, \Delta\psi_{1,t}, \Delta\psi_{2,t}, \Delta\psi_{3,t}, a_{L,t-1}, a_{R,t-1}) \tag{4.1}$$

where:

- (x_t, y_t): The robot's global position. In our experimental setup, the OptiTrack Motion Capture system publishes the JetBot's position (x, y) and orientation quaternion (q_x, q_y, q_z, q_w) at a 5 Hz rate. The position is taken as is, while the quaternion is used to compute the yaw angle θ, which is required for determining relative headings.
- $(d_{G,t}, \Delta\psi_{G,t})$: Euclidean distance and heading error (yaw difference) relative to the current goal, derived from the global pose described above.
- $\{d_{i,t}, \Delta\psi_{i,t}\}_{i=1}^{3}$: Distances and angular offsets to the three closest obstacle points, providing a local occupancy representation.

- $(a_{L,t-1}, a_{R,t-1})$: The previous action vector applied to the left and right motors.[1]

4.1.2 Action Space $\mathcal{A}$

The action space $\mathcal{A} \subset \mathbb{R}^2$ consists of continuous commands for the left and right motor voltages:

$$a_t = [u_{L,t}, u_{R,t}]^T \in [-0.6, 0.6]^2 \tag{4.2}$$

As detailed in Chapter 3, these values are hard-clipped within the environment to match the validated operational range of the physical actuators, preventing the "high-gain instability" observed in early Sim-to-Real trials.

4.1.3 Reward Function r

The reward function $r(s_t, s_{t-1}, a_t)$ is a weighted sum designed to guide the robot toward the goal while penalizing dangerous states. Following the structure in [3], we define:

Goal-reaching and Progress:

$$r_{goal}(s_t) = R_{goal} \cdot \mathbb{I}(d_{G,t} \leq d_{reached}) \tag{4.3}$$

$$r_{\Delta d}(s_t, s_{t-1}) = R_{\Delta d} \cdot (d_{G,t-1} - d_{G,t}) \cdot \lambda_{LOS} \tag{4.4}$$

where $\lambda_{LOS} \in \{0.2, 1.0\}$ is a visibility factor derived from the Logic Wrapper (Section 4.3), which discounts progress if the goal is occluded by obstacles.

Collision and Exploration: To penalize terminal failures and encourage coverage of the state space:

$$r_{coll}(s_t) = -R_{coll} \cdot \mathbb{I}(\text{Collision}) \tag{4.5}$$

$$r_{exp}(s_t) = R_{exp} \cdot \mathbb{I}(s_t \in \mathcal{S}_{unvisited}) \tag{4.6}$$

The exploration term r_{exp} provides a small dense reward for entering new grid cells, preventing the agent from remaining stationary in safe zones.

Regularization penalties: To ensure feasible sim-to-real transfer and prevent controller saturation, we implement two regularization terms based on the action magnitude:

$$r_{overpower}(a_t) = -P_{overpower} \cdot \sum_{i \in \{L,R\}} \max(0, |u_{i,t}| - 0.6)^2 \tag{4.7}$$

$$r_{center}(a_t) = -P_{center} \cdot \|a_t\|_2^2 \tag{4.8}$$

The **Overpower Penalty** ($r_{overpower}$) strictly penalizes the "forbidden" voltage range ($|u| > 0.6$). The **Centering Penalty** (r_{center}) prevents "Bang-Bang" behavior; without it, the policy tends to output excessively large logits (e.g., $u = 5.0$) relying on the clip, effectively saturating the controller and losing gradient sensitivity.

The total primal reward is computed as:

$$r^{primal} = r_{goal} + r_{\Delta d} + r_{coll} + r_{exp} + r_{overpower} + r_{center} \tag{4.9}$$

[1]Note: While included in the nominal state design to assist with temporal correlation, our ablation study (Section 5.6) later revealed that the LSTM can effectively infer the motion state even without these explicit action inputs.

4.1.4 Cost Function c and Safety Constraints

In addition to the reward, the environment computes a cost signal $c(s_t, a_t)$ to enforce safety. While [3] utilized a linear cost formulation, we implement a steeper **exponential barrier** to strongly discourage approaching walls:

$$c(s_t) = C_{obstacle} \cdot \sum_{i=1}^{3} \frac{\exp(k \cdot (1 - \frac{d_{i,t}}{d_O})) - 1}{\exp(k) - 1} \tag{4.10}$$

where d_O is the minimum safe distance. The objective is to maximize the cumulative reward subject to a safety budget C_{max}:

$$\max_{\pi} J_R(\pi) \quad \text{s.t.} \quad J_C(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_t \gamma^t c(s_t) \right] \leq C_{max} \tag{4.11}$$

4.2 Policy Architecture: Memory and State Estimation

A critical finding of this project was that domain randomization alone is insufficient if the agent lacks the temporal context to detect domain shifts (e.g., the sudden loss of traction). We investigated two architectural strategies to resolve this: Frame Stacking and Recurrent Neural Networks.

4.2.1 The Perceptual Aliasing Problem

Consider a robot navigating a slippery surface.

- **State A (Stopped):** The robot is stationary ($v = 0$). The visual inputs $\{d_{i,t}\}$ are static.
- **State B (Slipping):** The robot is applying max throttle ($u = 0.6$) but slipping on water. $v \approx 0$. The visual inputs are static.

To a memoryless Feedforward (MLP) policy, these two states are visually indistinguishable (Perceptual Aliasing). The agent perceives "I am not moving" but cannot differentiate between "I haven't started yet" and "I am slipping," leading to a policy that persistently outputs forward throttle without changing strategy.

4.2.2 Approach I: Frame Stacking (Finite Context)

Our initial attempt to resolve this ambiguity employed the standard **PPO algorithm** with **Frame Stacking**. We augmented the observation space to include a sliding window of the k most recent observations:

$$s_t^{stack} = [s_t, s_{t-1}, \ldots, s_{t-k+1}] \tag{4.12}$$

This formulation allows a standard Multi-Layer Perceptron (MLP) to implicitly estimate velocity and acceleration from the finite difference between stacked frames.

Limitations: While Frame Stacking successfully resolved short-term ambiguities, we found it insufficient for the "Stuck Robot" problem. The core issue is the **Finite Horizon**: if the robot remains immobilized for longer than k timesteps (where k is the stack size), the history buffer becomes filled with identical static observations. Consequently, the agent "forgets" that it was previously moving and reverts to the behavior of a memoryless policy, failing to trigger recovery maneuvers.

4.2.3 Approach II: Recurrent PPO (Infinite Context)

To address the limitations of the fixed window, we transitioned to the **Recurrent PPO** algorithm, which utilizes a **Long Short-Term Memory (LSTM)** network [2].

While a standard policy makes decisions based solely on the current snapshot of the world, the LSTM maintains a hidden state vector h_t that evolves over time. This acts as a memory, allowing the agent to base its decision on the recent history of observations $H_t = (s_{t-k}, \ldots, s_t)$.

Mechanism: Temporal Discrepancy Detection Functionally, the LSTM resolves the perceptual aliasing by detecting *motion discrepancies* over time.

- **Detection:** When the robot commands a forward voltage but the observations remain static for a sustained duration, the LSTM's internal state accumulates this mismatch.
- **Differentiation:** Unlike a memoryless policy that sees every static frame as an independent event, the LSTM differentiates between "stopped at start" (short duration) and "stuck in puddle" (long duration).
- **Reaction:** As the duration of the immobilization increases, the hidden state shifts, eventually triggering a policy shift to recovery behaviors (such as reversing or oscillating) to break traction.

4.2.4 Policy Interpretation: Implicit Context Adaptation

By combining this recurrent architecture with the multi-modal Domain Randomization described in Chapter 3, the agent learns to adapt to the underlying terrain dynamics.

During the initial interaction with a new surface, the LSTM implicitly estimates a **latent context** from the sequence of observations. If the robot fails to move despite applying throttle, this context shifts to represent a "low-control" regime. The policy head then conditions on this context to select a more aggressive or oscillatory control strategy, effectively adapting to the environment without explicit parameter estimation.

4.2.5 Training Hyperparameters

We utilized the `RecurrentPPO` algorithm from Stable-Baselines3, adapted for the JAX environment. Table 4.1 summarizes the key hyperparameters used, matching the exact configuration from our training script.

4.3 Logic-Based Safety Wrappers

While RL handles the local dynamics, we observed that the agent often fell into local minima (e.g., trying to drive through a wall to reach a goal behind it). To improve training efficiency, we implemented a deterministic **Line-of-Sight (LOS) Wrapper**.

Before passing the goal vector to the agent, the environment performs a ray-cast check.

$$g_{effective} = \begin{cases} g_{true} & \text{if RayCast}(p_{robot}, g_{true}) \text{ is clear} \\ g_{subgoal} & \text{otherwise} \end{cases} \tag{4.13}$$

[2]Due to the high parallelism of the JAX environment ($N_{env} = 4096$), the aggregate batch size is significantly larger than standard single-GPU implementations to ensure stable gradient estimation.

Parameter	Role	Value
H	Primal updates per dual (Lagrangian) update	200
α_λ	Lagrange multiplier learning rate	0.5
α_θ	Policy learning rate (Adam)	3×10^{-4}
C_{limit}	Constraint threshold (Upper bound for Cost)	0.02
P_{safe}	Constraint threshold (Lower bound for Safety Prob)	0.98
γ	Discount factor	0.99
c_{ent}	Entropy coefficient (Exploration)	0.05
$T_{rollout}$	Rollout steps per environment	32
B_{mini}	Aggregate Minibatch size[2]	65,536

Table 4.1: Hyperparameters utilized in the training process.

In our implementation, if the direct path to the goal is blocked by a wall, the goal distance $d_{G,t}$ in the reward function is artificially penalized by the factor λ_{LOS}. This "hysteretic" logic prevents the agent from getting stuck pushing against walls and encourages it to circumnavigate obstacles.

Chapter 5

Experiments and Results

This chapter presents the experimental evaluation of the Sim-to-Real pipeline. Unlike purely simulation-based studies, deploying reinforcement learning policies on low-cost hardware introduces significant stochasticity. Factors such as battery voltage fluctuation, motor heat, hardware discrepancies between individual robots, and uncontrolled variations in the experimental setup—specifically the precise saturation of the ground and the residual moisture on the wheels—create a high-variance testing environment.

Consequently, obtaining statistically significant quantitative metrics (e.g., "98.5% success rate") on the real robot proved infeasible due to the prohibitive time cost of running thousands of real-world trials and the inconsistency of hardware behavior across runs. Therefore, this chapter focuses on a qualitative analysis of the robot's behavior. We report observed failure modes, emergent recovery strategies, and the comparative stability of the different architectures (MLP, Frame Stacking, LSTM) and randomization strategies (UDR, GMM, Data-Driven).

5.1 Experimental Setup

5.1.1 Simulation Performance (The Reality Gap)

Before deploying to the real world, all policies (MLP, Frame Stacking, and LSTM) were trained until convergence in the JAX environment.

Observation: In simulation, all architectures achieved near-perfect success rates ($> 98\%$) across both Dry and Wet domain parameters. This indicates that the challenge is not the *solvability* of the task or the difficulty of the maze, but purely the Sim-to-Real transfer. The high fidelity of the simulation (using Data-Driven or GMM models) was sufficient to train competent agents, but the real-world deployment revealed robustness issues masked by the simulator.

5.1.2 Real-World Scenarios

We evaluated the robot in the Sycamore Lab maze under three conditions:

1. **Standard Dry Ground:** The default lab floor. Represents the "Training Distribution" for UDR.

2. **Rough Terrain:** Sandpaper and plastic. Used to test robustness to vibration and minor friction changes.

3. **Wet/Slippery Conditions:** To reproduce the "Out-of-Distribution" failure mode (Cluster 0 from Chapter 3), we physically altered the traction conditions by saturating the robot's tires with water or creating localized puddles directly on the standard maze floor. This setup induces high deadzones and unpredictable loss of control authority, effectively simulating the "Stuck" regime.

5.2 Phase I: The Baseline (MLP + Uniform Randomization)

The first set of experiments evaluated the baseline Feedforward (MLP) architecture trained with Uniform Domain Randomization (UDR).

5.2.1 Performance on Dry Ground

On standard dry surfaces, the MLP policy performed adequately. The robot successfully navigated the maze, avoided walls using the distance observations, and reached the goal.

Validation of Logic Wrappers: The Line-of-Sight (LOS) wrapper proved critical for resolving geometric local minima. In early iterations, the agent frequently stagnated in the top-left corner of the maze, attempting to drive through a wall to reach the goal. The LOS wrapper successfully eliminated this failure mode by penalizing the occluded goal vector, forcing the agent to circumnavigate the obstacle.

Minor Limitation: While the trajectory was logical, we observed that the agent still exhibited high-frequency oscillations in narrow corridors. This is likely due to the "Bang-Bang" nature of the clipped control actions described in Chapter 4, rather than a failure of the navigation logic itself.

5.2.2 Failure Mode: The "Spinning" Robot

When introduced to the Wet/Puddle scenario, the MLP agent failed catastrophically. This failure stems from a combination of Out-of-Distribution (OOD) physics and architectural limitations.

Observed Behavior: Upon entering the slippery patch, the drastic reduction in traction caused the wheels to spin in place.

- **Training Bias (UDR):** Since the Baseline was trained via Uniform Domain Randomization on the "Dry" regime ($\alpha \in [45, 55]$), it learned a strong prior that a moderate voltage (e.g., $u = 0.5$) *always* results in forward motion.

- **Physical Mismatch:** On the wet surface, this assumption breaks. The wheels spin, but the chassis remains stationary ($v_{robot} = 0$).

- **Perceptual Aliasing:** Crucially, the memoryless MLP cannot detect this mismatch. It perceives the static sensor readings and, relying on its "Dry" training, assumes it simply hasn't applied enough throttle yet—or worse, assumes it is moving and the sensor is static.

- **Outcome:** The policy persists in outputting the optimal "Dry" action, resulting in continuous ineffective wheel spin until timeout.

This highlights a dual failure: the UDR strategy failed to prepare the agent for the low-friction regime, and the memoryless architecture prevented the agent from detecting that its internal model of the dynamics was incorrect.

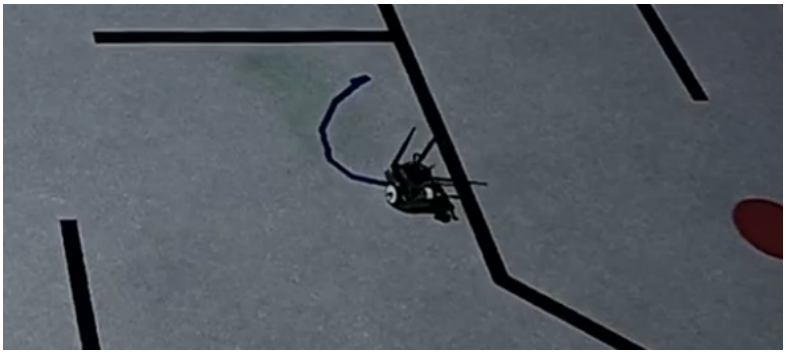

Figure 5.1: **MLP Failure on Wet Surface:** The robot achieves minimal displacement (short dark blue trajectory) before losing traction. Despite the wheels spinning, the chassis remains stationary, and the agent fails to reach the goal (red circle on the right), illustrating the failure of the "Dry" policy on wet terrain.

As illustrated in Figure 5.1, the physical displacement is negligible. The short trace indicates the brief initial motion before the wheels lost grip. Once traction was lost, the agent could not adapt its strategy, the wheels kept spinning and the robot was immobile.

5.3 Phase II: Adding Temporal Context (Frame Stacking)

To resolve the "Spinning" robot issue, we introduced Frame Stacking (history of $k = 3$ frames). We evaluated this architecture with both GMM-based and Data-Driven physics models to see if precise friction sampling could stabilize the behavior.

5.3.1 Behavioral Analysis

As shown in Figure 5.2, this configuration yielded a tangible improvement over the baseline but failed to provide a robust solution. We observed no significant performance difference between the GMM and Data-Driven variants of the Frame Stacking policy; both suffered from the same limitations.

- **Partial Success (Physics Model Effect):** Unlike the UDR-trained MLP which failed immediately, both the GMM and Data-Driven agents learned to output higher torques, allowing them to "power through" initial slips. The robot successfully navigated the majority of the maze.
- **Jerky Motion:** The blue trajectory trace in Figure 5.2 reveals severe oscillations. The policy is aggressive, constantly correcting its heading. This suggests that while the agent can generate movement, it lacks a smooth internal model of the sliding dynamics, leading to a "struggling" motion profile.
- **The Finite Horizon Limit:** The failure occurs at the end of the run. When the robot hit a particularly difficult patch near the goal, its progress slowed. Once the immobilization

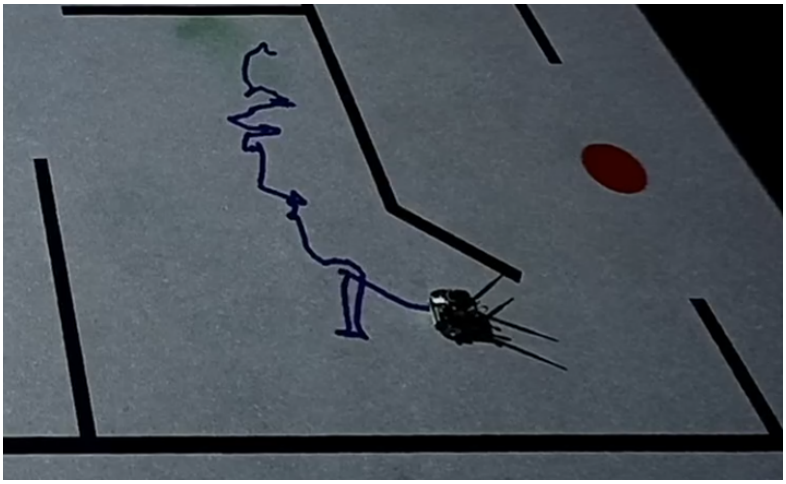

Figure 5.2: **Frame Stacking + GMM Performance:** Unlike the MLP, this agent manages to traverse a significant portion of the wet terrain, proving that the physics-informed training helped adapt to the low-friction regime. However, the trajectory (blue line) is highly irregular and "jerky," indicating a constant struggle for traction. Ultimately, the robot becomes permanently stuck near the goal (red circle).

duration exceeded the stack window ($k = 3$ frames), the history buffer filled with static observations ($s_t \approx s_{t-1}$). At this point, the agent lost its temporal context, reverted to the behavior of a memoryless policy, and failed to trigger further recovery maneuvers, remaining indefinitely stuck.

Conclusion: Frame Stacking combined with advanced physics modeling extends the robot's range but is ultimately brittle. It pushes the failure point further down the track but cannot solve long-duration immobilization.

5.4 Phase III: Recurrent Policy (LSTM)

The final iteration employed the LSTM architecture trained with the Data-Driven (Non-Parametric) physics model. As shown in Figure 5.3, this configuration was the only one to achieve the full objective.

5.4.1 Emergent Recovery Behavior (Adaptive Re-planning)

On the Wet/Slippery surface, the LSTM agent demonstrated a capability absent in previous architectures. When the robot lost traction and the chassis stopped moving:

1. **Detection:** The robot would experience wheel spin for a duration of several seconds (typically 2-4 seconds) while the LSTM state integrated the lack of visual progress.
2. **Response:** Unlike the MLP (which maintained a constant failed action), the LSTM agent would visibly **alter its strategy**. This was not limited to a single behavior; sometimes it would attempt a high-frequency oscillation ("wiggling"), while other times it would suddenly reverse direction or aggressively drive the opposite wheel to pivot the chassis.

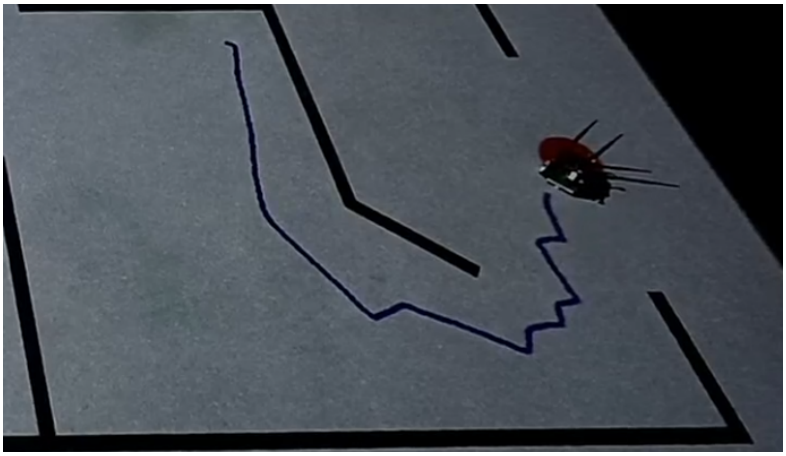

Figure 5.3: **LSTM + Data-Driven Success:** This trajectory (blue line) demonstrates a successful Sim-to-Real transfer. Unlike previous architectures, the robot traverses the entire wet maze and successfully reaches the goal (red circle). The path is notably smoother than the Frame Stacking attempt, with fewer erratic corrections.

3. **Outcome:** This varied behavioral search often altered the contact dynamics enough to find traction. By cycling through different action primitives, the agent increased the probability of escaping the low-friction patch.

This confirms that the temporal context provided by the LSTM allowed the network to learn a latent representation of "Stuck," triggering a transition to a diverse set of recovery sub-policies. Furthermore, the use of Data-Driven sampling (directly sampling from the system identification logs rather than a fitted GMM) likely provided the policy with more realistic "worst-case" friction samples during training.

5.4.2 Stability and Inconsistency

Despite the qualitative success shown in Figure 5.3, the LSTM solution is not consistently robust. While it represents the state-of-the-art for this project, it is not a "solve-all" policy.

- **Stochastic Success:** In trials with identical setups (same start position, same puddle), the outcomes varied. The robot would sometimes successfully wiggle free and reach the goal, while in other attempts its wheels would keep spinning ineffectively until timeout.
- **Rare Failures on Dry Ground:** Counter-intuitively, we observed rare failure cases even on standard dry terrain. In specific map configurations, the LSTM would occasionally trigger its "recovery" behavior (wiggling) falsely, likely hallucinating a stuck state due to sensor noise or specific wall geometries.
- **Sensitivity to Placement:** The policy displayed hypersensitivity to initial conditions. A slight variation in the robot's starting position or its entry angle into a corridor could determine success or failure. This suggests the policy has learned specific "successful trajectories" rather than a fully generalized robust control law.
- **Inconsistent Wet Performance:** Even with wet wheels, the performance was not guaranteed. While the "wiggling" behavior often emerged, there were instances where the robot

would simply get stuck again in a different part of the map, despite having successfully escaped a similar situation moments earlier.

5.5 Ablation Study: Architecture and Observation Space

To isolate the source of the performance improvement, we compared different input configurations for the LSTM policy. Specifically, we tested whether explicitly including the previous action vector a_{t-1} in the observation s_t was necessary for detecting the "stuck" condition.

5.5.1 Implicit vs. Explicit Action History

We compared two LSTM configurations:

1. **Explicit History:** The standard configuration where $s_t = [Observations, a_{t-1}]$.
2. **Implicit History:** A reduced configuration where $s_t = [Observations]$ only.

Finding: We observed no significant qualitative difference in performance between these two configurations on the real robot. Both configurations successfully triggered the "adaptive re-planning" behaviors when stuck, and both exhibited similar rates of instability.

Interpretation: This suggests that the LSTM does not strictly require the explicit value of the previous voltage to detect immobilization. The network likely infers the "Stuck" state purely from the temporal persistence of static observations. If the observations remain identical ($s_t \approx s_{t-1}$) for an extended duration, the recurrent update equations drive the hidden state h_t toward a specific region of the latent space, signaling that the current strategy is ineffective.

5.6 Analysis of Physics Modeling (GMM vs. Data-Driven)

Throughout Phases II and III, we compared policies trained with the GMM (Parametric) model versus the Data-Driven (Non-Parametric) replay buffer.

Observation: Across both architectures (Frame Stacking and LSTM), we found no significant qualitative difference in real-world performance between these two randomization strategies.

- **Frame Stacking:** Both GMM and Data-Driven variants resulted in the same "jerky" behavior and eventual finite-horizon failure.
- **LSTM:** Both GMM and Data-Driven variants successfully learned the emergent recovery behaviors (re-planning/wiggling).

This suggests that the critical factor was shifting the mean of the training distribution to cover the "Wet/High-Deadzone" regime (which both models achieve), rather than the specific shape or granularity of the noise distribution. As long as the agent is exposed to the low-friction physics during training, the bottleneck for Sim-to-Real transfer becomes the architecture's capacity (Finite vs. Infinite Memory) to handle the resulting partial observability, rather than the precision of the simulation model itself.

This also serves as a validation of our Piecewise Affine modeling choice. The fact that the parametric GMM performed on par with the raw data replay suggests that the GMM successfully captured the core stochastic properties of the actuators (the deadzone and gain distributions) without needing to store the entire raw dataset.

5.7 Summary of Experimental Findings

The real-world deployment highlights several key takeaways that were not apparent in simulation:

1. **Memory is Essential:** The memoryless MLP is fundamentally incapable of handling friction-based immobilization. No amount of domain randomization can fix the perceptual aliasing problem.

2. **Infinite Context ¿ Finite Context:** Frame Stacking is insufficient for real-world recovery because "stuck" events often outlast the stack window. The LSTM's infinite horizon is required to trigger persistent recovery attempts.

3. **Instability Remains:** While the LSTM approach represents the best-performing configuration, it is not a "solved" system. The robot remains prone to inconsistent behaviors. Factors such as battery voltage fluctuation, motor heat, hardware discrepancies between individual robots, and uncontrolled variations in the experimental setup, specifically the precise saturation of the ground and the residual moisture on the wheels—create a high-variance environment that is not currently captured even in our high-fidelity JAX simulation.

Chapter 6

Conclusion and Future Work

In this semester project, we addressed the Sim-to-Real transfer problem for low-cost differential drive robots, specifically focusing on the critical failure mode of immobilization on low-friction surfaces. By analyzing the physical limitations of the JetBot platform, we transitioned from standard feedforward architectures to recurrent policies capable of adaptive re-planning.

While we did not achieve a universally robust "solve-all" policy, our experiments demonstrate that equipping the agent with temporal context (LSTM) allows for the emergence of recovery behaviors that are impossible for memoryless agents. This significantly improves the robot's resilience to unmodeled friction patches, even if the final behavior remains stochastic.

6.1 Summary of Contributions

Our work makes three primary technical contributions to the Sycamore Lab's robotics framework:

1. **High-Throughput JAX Infrastructure:** We successfully migrated the training pipeline to a JAX-based environment. This allowed for the parallel simulation of 4,096 agents, enabling the use of large-scale PPO updates which stabilized the training of recurrent policies and reduced the experimentation cycle from days to hours.

2. **Physics-Aware Domain Randomization:** We demonstrated that the failure of standard baselines (UDR) was driven by a distribution mismatch—specifically, the lack of "Wet/High-Deadzone" samples during training. By performing System Identification and incorporating these failure modes into the training distribution (via GMM or Data-Driven sampling), we forced the agent to learn high-torque control policies capable of overcoming stiction.

3. **Implicit Context Adaptation via LSTM:** We identified that the primary cause of the "Spinning" robot failure was perceptual aliasing. By transitioning to Recurrent PPO, we enabled the agent to function as a temporal integrator. The resulting policies exhibit emergent "wiggling" and re-planning behaviors, effectively performing implicit system identification to break traction, a capability that emerged without explicit reward engineering.

6.2 Limitations

Despite the qualitative improvements, the proposed pipeline is not yet a fully reliable deployment solution. Several critical limitations remain:

- **Stochasticity and Hardware Variance:** As detailed in Chapter 5, the "wiggling" recovery behavior is inconsistent. The success of the transfer is heavily influenced by unmodeled hardware variables, including battery voltage sag, motor heat accumulation, and subtle variations in wheel moisture. The current simulation, despite its fidelity, does not capture these transient internal states.
- **False Positives (Instability on Dry Ground):** The LSTM policy occasionally hallucinates a "stuck" state on standard dry terrain, leading to unnecessary oscillations or "twitchy" behavior in narrow corridors. This suggests that the policy may be over-fitting to the recovery objective, prioritizing movement over smoothness.
- **Sensitivity to Initial Conditions:** The probability of successful recovery is hypersensitive to the robot's entry angle and speed into a slippery patch. This indicates that the policy has likely learned specific successful trajectories rather than a generalized, robust control law for low-friction dynamics.

6.3 Future Work

Based on these findings, we propose the following directions to address the remaining instability:

6.3.1 Explicit Meta-Reinforcement Learning

While the LSTM adapts implicitly, it is opaque. Explicit Meta-RL algorithms (like MAML or PEARL) could be employed to infer a latent context variable z representing the current physics (e.g., $z \approx$ friction coefficient). This would allow the policy to explicitly switch between "Dry Mode" (smooth driving) and "Wet Mode" (high-torque wiggling), potentially reducing the false positives on dry ground.

6.3.2 Online System Identification

To address the hardware variance (battery/heat), an Online SysID module could be implemented. A parallel estimator (e.g., an Unscented Kalman Filter) could estimate the motor efficiency constants in real-time. Feeding this estimate into the policy would allow the agent to adjust its control gains dynamically as the battery drains or motors heat up.

6.4 Closing Remarks

The "Sim-to-Real" gap is often framed as a problem of making the simulation look more like reality. Our results suggest that for low-cost robots, perfectly modeling the environment is likely impossible due to the chaotic nature of cheap actuators. Instead, the path forward lies in making the *agent* resilient to its own physical limitations. By acknowledging that the robot will inevitably encounter unmodeled dynamics, and equipping it with the memory to detect and react to these failures, we move closer to truly autonomous robotic systems.

Bibliography

[1] Yevgen Chebotar, Ankur Handa, Viktor Makoviychuk, Miles Macklin, Jan Issac, Nathan Ratliff, and Dieter Fox. Closing the sim-to-real loop: Adapting simulation randomization with real world experience. In *International Conference on Robotics and Automation (ICRA)*, 2019.

[2] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[3] Federico Rocca. Safe reinforcement learning for robot maze escaping. Semester project report, École Polytechnique Fédérale de Lausanne (EPFL), 2025. Unpublished.

[4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[5] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017.